

TUPLES & IMMUTABLE COLLECTIONS

Block 18 • Fixed groups of related values

A tuple is an ordered collection that is meant to stay fixed.

```
location = (30.4, -86.9)
```



30.4

-86.9

latitude

longitude

From lists to tuples

Both are ordered. Only one is designed to change.

```
temperatures = [72.5, 75.1, 81.2]
```

```
location = (30.4213, -86.6170)
```

List mindset

A changing collection:

- add values
- remove values
- update positions

Tuple mindset

A fixed grouping:

- fixed count
- meaningful positions
- should not be modified

Creating a tuple

Parentheses group related values in a fixed order.

```
location = (30.4, -86.9)

dimensions = (1920, 1080)

rgb_color = (20, 221, 230)
```

A tuple is similar to a list because it is:

- ordered
- indexed
- sliceable

But after creation:

You cannot change, append, insert, or remove values in place.

Ordered and indexed

Tuple positions work like list positions and string positions.

```
location = (30.4, -86.9)

print(location[0])
print(location[1])
print(location[-1])
```

30.4

-86.9

index 0

index 1

-1

Zero-based indexing carries forward.

Negative indexing counts from the end.

Tuple unpacking

Give meaningful names to positions.

```
location = (30.4, -86.9)

latitude, longitude = location

print(latitude)
print(longitude)
```

Before unpacking

location[0]
location[1]



After unpacking

latitude
longitude

Unpacking turns positional meaning
into readable variable names.

Another unpacking example

Width and height form a fixed pair.

```
dimensions = (1920, 1080)

width, height = dimensions

total_pixels = width * height

print(total_pixels)
```

`dimensions[0]` is not wrong.

But width is clearer than “position zero.”

Fixed pairs appear everywhere:

- width, height
- min, max
- latitude, longitude

Slicing tuples

Tuples support the same start:end slicing pattern.

```
values = (10, 20, 30, 40, 50)
```

```
first_three = values[0:3]
```

```
middle_values = values[1:4]
```

10

20

30

40

50

0

1

2

3

4

```
values[0:3]
```

returns a new tuple:

```
(10, 20, 30)
```

Immutability

A tuple cannot be changed in place.

```
dimensions = (1920, 1080)  
  
dimensions[0] = 1280
```

This fails because the tuple itself is immutable.

Important idea

If you need different values, create a new tuple or use a list when changes are expected.

BLOCK 18

Mutable versus immutable behavior

Lists change. Tuples reject in-place changes.

```
list_values = [10, 20, 30]
list_values[0] = 99

print(list_values)
```

List result

[99, 20, 30]

```
tuple_values = (10, 20, 30)
tuple_values[0] = 99

# TypeError
```

Tuple result

Python refuses the
change.

Why use a tuple?

Use a tuple when position has meaning and the group is fixed.

Good tuple candidates

- coordinates
- width and height
- minimum and maximum
- RGB color components
- fixed database rows

Less ideal tuple candidates

- validation errors
- temperatures over time
- record IDs to process
- filenames being collected

Plain-language test

“Would changing the number or order of values change what this object means?”

Lists versus tuples

The practical beginner distinction: changing collection versus fixed grouping.

Feature	List	Tuple
Ordered	Yes	Yes
Indexed	Yes	Yes
Can change	Yes	No
Add/remove	Yes	No
Common use	Changing collection	Fixed grouping

Avoid the rabbit hole

Performance differences are not the point today. The point is choosing a structure that communicates intent.

A tuple generally says: this group should stay together and stay the same size.

Tuples as configuration pairs

A tuple can keep related boundary values together.

```
temperature_range = (60, 80)

minimum_temperature, maximum_temperature =
    temperature_range

temperature = 72

is_valid = minimum_temperature <= temperature <=
    maximum_temperature
```

The tuple groups the rule:

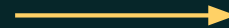
acceptable low and high
temperature.

Then unpacking makes the
comparison readable.

Returning multiple values conceptually

Later, functions can hand back grouped results.

```
def get_dimensions():  
    width = 1920  
    height = 1080  
    return width, height  
  
width, height = get_dimensions()
```



Conceptual idea

Multiple related values can travel together as a tuple-like result.

No need to master functions yet.

Just recognize why fixed groups are useful.

BLOCK 18

Coordinates and structured pairs

Order gives meaning: latitude first, longitude second.

```
navarre = (30.4213, -86.6170)

latitude, longitude = navarre

print(f"Lat: {latitude}")
print(f"Lon: {longitude}")
```

Order matters

(30.4213, -86.6170)

is not the same meaning as

(-86.6170, 30.4213)

**A tuple helps keep the pair
together.**

Guided exercise 1: coordinates

Create, index, unpack, and print a formatted coordinate.

```
location = (30.4213, -86.6170)

# 1. Print latitude
# 2. Print longitude
# 3. Unpack the tuple
# 4. Print a formatted coordinate
```

Expected habits

- predict positions first
- use meaningful names
- format the output for humans

```
latitude, longitude = location

print(f"Location: {latitude},  
{longitude}")
```

Guided exercise 2: allowed range

Use a tuple to store minimum and maximum values.

```
temperature_range = (60, 80)

minimum_temperature, maximum_temperature =
temperature_range

temperature = 72

is_valid = minimum_temperature <= temperature <=
maximum_temperature

print(is_valid)
```

Test values

- 59
- 60
- 72
- 80
- 81

Boundary testing checks the edges of the rule.

Guided exercise 3: screen resolution

Unpack dimensions, then calculate total pixels.

```
resolution = (1920, 1080)

width, height = resolution

total_pixels = width * height

print(f"Width: {width}")
print(f"Height: {height}")
print(f"Pixels: {total_pixels}")
```

Result

$1920 \times 1080 = 2,073,600$
pixels

Connection

This ties back to images as
grids of numeric pixel values.

Guided exercise 4: list vs tuple behavior

Run both examples and describe the difference.

```
list_values = [10, 20, 30]
list_values[0] = 99
print(list_values)
```

```
tuple_values = (10, 20, 30)
tuple_values[0] = 99
print(tuple_values)
```

Discussion

Which one changes?

Which one raises an error?

What does that tell us about mutability?

Read the final error line.
It usually tells you the important clue.

Applied challenge: fixed configuration values

Represent validation ranges as tuples and reuse them.

```
acceptable_temperature = (60, 80)
acceptable_voltage = (110, 125)
acceptable_completion = (0, 100)
```

Program tasks

1. Unpack each pair
2. Store example measurements
3. Validate each value
4. Print a readable summary

Purpose

Configuration values can be grouped once and reused wherever the rule is needed.

Challenge starter code

A clear pattern: unpack, compare, report.

```
temp_low, temp_high = acceptable_temperature
volt_low, volt_high = acceptable_voltage
comp_low, comp_high = acceptable_completion

temperature = 74
voltage = 121
completion = 98

temperature_ok = temp_low <= temperature <= temp_high
voltage_ok = volt_low <= voltage <= volt_high
completion_ok = comp_low <= completion <= comp_high
```

Then print every result.

Avoid hiding the logic in
one giant expression.

**Readable code beats
clever code.**

Common mistakes

Most tuple bugs come from count, order, or trying to mutate.

Unpacking mismatch

```
latitude, longitude, altitude =  
location
```

Too many names for two values.

Order confusion

```
(width, height)
```

is not the same as

```
(height, width)
```

Mutation attempt

```
dimensions[0] = 1280
```

Tuples do not support item assignment.

Debugging move: print the tuple and print `len(tuple)`.

Mini knowledge check

Students should answer in plain language.

```
dimensions = (1920, 1080)

width, height = dimensions

print(dimensions[0])
print(dimensions[-1])
print(width * height)
```

Predict

1. First output
2. Second output
3. Third output
4. Why use a tuple here?

Answers

```
1920
1080
2073600
```

Because width and height are a fixed pair.

Wrap-up: choose the right collection

Lists and tuples are both useful, but they say different things.

Use a list when values may be collected, changed, added, or removed.



Use a tuple when values form a fixed group with meaningful positions.

```
validation_errors = []           # list
location = (30.4, -86.9)         # tuple
range_ok = (60, 80)              # tuple
```